

목차

- 01 바이브 코딩이란?
- 02 나에게 필요한 기능 찾기
- 03 프롬프트 빌딩
- 04 실제 기능 구현하기
- 05 결과물 확인 및 QA
- 06 숙제

01

바이브코딩의 정의

바이트코딩이란?

자연어로 설명하고, AI가 만들고,
실행해서 확인하는 개발 방식

2025. 02

Andrej Karpathy(안드레이 카파시)가 처음 이 말을 썼습니다.

원래 정의는 극단적이었습니다.

나온 코드는 거의 읽지 않고, 동작하기만 하면 된다는 태도.

**“에러가 나면 메시지만 복사해서 다시 던진다.
대부분 그러면 고쳐진다.”**

지금은 이 네 단계의 반복을 말합니다

01

설명

원하는 결과를 자연어로 말한다.

02

생성

AI가 코드를 만든다.

03

확인

실행해서 결과를 본다.

04

재요청

다시 말로 수정을 요청한다.

LLM — 다음에 올 말을 예측하는 모델

- Large Language Model, 대규모 언어 모델. 방대한 텍스트와 코드를 학습했습니다.
- 하는 일은 하나입니다 — 주어진 말 다음에 올 말을 확률로 예측해 이어 씁니다.
- 코드도 결국 텍스트이므로, 같은 방식으로 쓰고 고칠 수 있습니다.
- ChatGPT, Claude, Gemini는 모두 이 모델 위에 올라간 서비스입니다.

프롬프트 — AI에게 건네는 입력 전체

요청

무엇을 만들까

원하는 결과물을 동작 단위로 말합니다. “버튼을 누르면 목록이 정렬된다”처럼.

맥락

어떤 상황인가

누가 쓰는지, 지금 코드가 어떤 상태인지, 어디에 붙는지를 함께 줍니다.

제약

무엇은 하지 말까

쓰지 말 기술, 지켜야 할 형식, 건드리지 말 부분을 미리 못 박습니다.

IDE 코딩과 프롬프트 코딩

항목	기존 IDE 코딩	프롬프트 코딩
알아야 하는 것	문법, 라이브러리, API	내가 뭘 원하는지, 구체적으로
작업 단위	함수·모듈을 직접 작성	결과물 단위로 요청 → 검토 → 재요청
디버깅	코드를 읽고 원인을 찾음	에러 메시지나 증상을 설명
속도	느리지만 통제 가능	빠르지만 과정 이해는 얇음
결과물 이해도	내가 짠 코드라 구조를 압	동작하지만 설명은 어려울 수 있음

왜 지금 가능해졌나

- LLM이 코드 생성과 수정을 충분히 잘하게 됐다.
- 챗 인터페이스 덕분에 설명 → 생성 → 확인 → 재요청이 몇 초 단위로 돈다.
- 예전엔 코드를 읽고 어디를 건드릴지 알아야 했지만, 지금은 증상만 설명해도 수정된다.

실패는 보통 두 지점에서 생깁니다

시작할 때

요청이 모호하다

“예쁘게 해줘”처럼 말하면 매번 다른 결과가 나옵니다.
기준을 먼저 정해야 같은 결과로 수렴합니다.

문제가 생겼을 때

증상을 설명하지 못한다

무엇이 어떻게 잘못됐는지 말하지 못하면 AI도 고치지 못합니다.
재현 조건까지 말해야 합니다.

**차이는 코딩 실력이 아니라, 원하는 걸 구체적으로
쫓아서 말할 수 있는가입니다.**

잘 맞는 범위와 아직 어려운 범위

잘 맞는다

개인용 도구와 프로토타입

혼자 쓰는 간단한 프로그램, 빠르게 확인해야 하는 아이디어.
틀려도 비용이 작은 영역입니다.

아직 어렵다

보안, 성능, 여러 서비스 연동.

“던지고 되나 보기”로는 잘 되지 않고, 이해한 사람이 필요합니다.

복잡한 시스템

쪼개서 말하기

코딩 실력이 아니라, 원하는 걸 나눠 설명하는
능력이 결과를 가릅니다.

02

나에게 필요한 기능 찾기

나는 무슨 문제에 갇혀있나요?

시작이 안 된다

- **시작을 못 한다**
몸이 안 움직여 30분씩 날린다
- **뭘 먼저 할지 못 정한다**
할 일 5개를 고르다 하루가 간다
- **목표를 너무 크게 잡는다**
“한 화 완성”을 적고 시작을 못 한다
- **잡일에 시동이 안 걸린다**
세금·메일·정리를 몇 주씩 미룬다

집중이 흩어진다

- **SNS에 빨려 들어간다**
자료 찾다 1시간 뒤에 정신 차린다
- **얼마나 걸릴지 모른다**
2시간이라 본 게 늘 6시간 걸린다
- **남 일에 내 작업이 밀린다**
외주·연락에 내 것은 손도 못 댈다
- **일이 안 끝난다**
퇴근이 없어 쉬어도 쉰 것 같지 않다

지속이 안 된다

- **아무도 안 시키면 안 한다**
마감이 없으면 루틴이 무너진다
- **하루 빼먹으면 다 놓는다**
한 번 빼먹으면 아예 안 열게 된다
- **밤낮이 바뀐다**
기상 시간이 계속 뒤로 밀린다

결과가 안 보인다

- **결과물이 없다**
하루 종일 앉아 있었는데 뭘 했는지 모른다
- **밤마다 허무하다**
“난 오늘 뭘 했지” 하고 자책하며 잔다
- **혼자라 피드백이 없다**
잘 되고 있는 건지 아닌지 모른다

03

프롬프트 빌딩

프롬프트를 만드는 과정

■ 01 문제 + 뭘 만들지

“뭘 먼저 할지 모르겠어서 시작을 못 하는 병에 걸렸어...
이럴 때 도움되는 웹페이지 뭘 만들면 좋을까? html로 만들거고 나만 쓸
거야.”

■ 02 구성요소로 바로

“바이브코딩 할 거라 기술 공부는 안 해.
어떤 화면이 있고 뭐가 들어가야 하는지만 얘기하자.”

■ 03 내 조건

“타이머는 15분짜리가 좋고 깔끔한 디자인”

■ 04 기능 캐기 ①

“입력한 할 일 추가·삭제·수정 되나?”

■ 05 기능 캐기 ②

“완료한 거 아카이빙 되나?”

■ 06 오류

“이런 거 처음 만들면 어떤 오류가 잘 나?”

■ 07 요약

“지금까지 나온 얘기 명세서로 정리해줘.”

■ 08 프롬프트

“지금까지 정한 거 + 저 오류 예외처리까지 넣어서 상세 프롬프트로 써줘.”

04

실제 기능 구현하기

05

결과물 확인 및 QA

QA란? 결과물 괴롭히기!

- **QA — Quality Assurance, 품질 보증**

만든 것이 제대로 작동하는지 확인하고, 문제를 찾아 고치는 일.

- **잘 되는지 확인하는 게 아니다**

안 되는 걸 찾으러 가는 것.

사용자가 제일 이상하게 굴 법한 짓을 내가 먼저 해본다.

- **한 번에 하나만 고친다**

고치고 나서 여섯 개를 다시 돌린다.

- **일부러 괴롭혀보기**

아무것도 없는 상태에서 눌러보기 · 같은 버튼 빨리 여러 번 누르기

- **말도 안 되는 값 넣기**

엄청 긴 글자, 0, 빈칸

- **환경을 흔들어보기**

새로고침(F5) · 껏다 다시 켜기 · 폰으로 열어보기

오류가 나오면 성공이다 — 못 찾은 건 잘 만든 게 아니라, 아직 안 눌러본 것.

06

숙제